OmniDesk Copilot

Real-Time AI Customer Support Intelligence & Coaching Platform

Version 2.1.0 | Groq LPU Sub-Second (<0.4s) | React 18 + Vite SPA | FAISS Dense Vector KB | Streamlit Cloud Embedded

Live Deployed Platform: https://customer-support-agent12.streamlit.app/

GitHub Repository: https://github.com/Guptanshu44/Customer-support-assistant.git

Core AI Models: Groq LLaMA 3.3 70B & Anthropic Claude 3.5 Sonnet

Vector Pipeline: FAISS FlatL2 + sentence-transformers/all-MiniLM-L6-v2

1. Executive Summary & Problem-Solution Matrix

Customer support specialists in enterprise environments face persistent operational challenges: high cognitive strain, inconsistent empathy under pressure, risk of SLA/policy compliance violations, slow retrieval across scattered wiki knowledge bases, and agent burnout.

OmniDesk Copilot solves these challenges by functioning as an active, in-flight pair coach that evaluates inbound customer queries and draft agent responses in **<0.4 seconds** using custom Groq Language Processing Units (LPUs).

Enterprise Challenge	Traditional Approach	OmniDesk Copilot Solution
Cognitive Fatigue & Burnout	Post-shift retrospective forms, unmonitored agent stress	Real-time `AgentBurnoutDetector` computing turn-by-turn burnout index (0–100) with supervisor rotation alerts
Robotic / Unempathetic Replies	Rigid static script templates and manual periodic audits	In-flight scoring (Tone, Empathy, Clarity) and context-aware enterprise reply suggestions with 1-click apply
SLA & Compliance Infractions	Accidental guarantees (e.g. unauthorized refunds outside 30-day window)	Proactive compliance guardrail rules evaluated before message dispatch with warning banners and remedies
Slow Knowledge Retrieval	Agents spend 2–4 minutes searching static PDF/Notion manuals	Dense 384d FAISS semantic vector retrieval surfacing relevant policy clauses in **<10ms**
Escalation Unpredictability	Disputes escalate unexpectedly without early warning signs	Turn-by-turn `ConversationMomentumForecaster` predicting resolution/escalation with up to 95% confidence

2. System Architecture & Dual-Runtime Architecture

OmniDesk Copilot implements a resilient dual-runtime architecture:

- **Standalone Full-Stack Architecture:** React 18 frontend communicating over REST API to Flask server (`server/app.py` on port 5000), using SQLite3 for local persistence and FAISS for vector search.
- **Streamlit Cloud Zero-Infra Deployment:** React 18 single-page application compiled into a self-contained single file bundle (`frontend/dist/index.html`) using `vite-plugin-singlefile` and mounted via `st.components.v1.html` with Cloud Firestore real-time synchronization.

3. Core Intelligence Engines Deep-Dive

3.1 Professional Enterprise AI Reply Engine (`coach.py` & `client.js`)

Completely redesigned prompt architecture enforcing **6 strict enterprise hard rules**:

1. **Strict Plain Text:** No markdown, bold text (`**`), asterisks, bullet points, or headers.
2. **Zero Emojis:** Strictly zero emojis under any circumstances.
3. **No Verbatim Echoing:** Never repeats customer sentences back to them.
4. **No Generic Filler:** Strips phrases like "*I would be delighted*", "*I understand your frustration*", or "*Great question*".
5. **Multi-Turn Greeting Awareness:** Suppresses greeting re-starts ("*Hello*", "*Hi*") if conversation is already in progress.
6. **Context-Driven Escalation:** If the customer confirms prior troubleshooting failed, skips repetitive questions and immediately advances to next diagnostic or escalation action.

Scenario	Generic Chatbot Response	OmniDesk Copilot Enterprise Response
Double Billing Dispute	`**Oh no!** 😊 I am so sorry to hear you were double charged! Let me help you with that right now!`	`Thank you for reaching out. Let me review your transaction records to verify and reverse the duplicate charge immediately.`
Repeated Technical Failure	`Have you tried turning your device off and on again? 🔌 Please let me know!` * (Customer already stated they did)*	`Since restarting the equipment did not restore connection, let us run an upstream diagnostic from our end to check the node status.`

3.2 Agent Burnout Detector (`coaching_assistant/burnout_detector.py`)

- Monitors sentiment trajectory, response length shrinkage, cognitive strain, and exposure to hostility.
- Computes **Burnout Index (0–100)** categorized into **Low** (0–39), **Medium** (40–69), and **Critical** (70–100).
- **Server Restart Persistence:** `_bootstrap_from_db()` replays stored turns through `observe()` on startup so burnout baselines survive server restarts.

3.3 Conversation Momentum Forecaster (`coaching_assistant/momentum_forecaster.py`)

- Calculates slope trajectories between customer hostility/urgency and agent empathy/clarity.
- Predicts outcome: `resolution`, `escalation`, `stalemate`, or `too_early` (<2 turns).
- Provides confidence score (up to 95%) and estimated turns remaining.
- **Server Restart Persistence:** Stored turns are replayed into `record_turn()` during bootstrap.

3.4 AI Micro-Habit Coach (`coaching_assistant/habit_coach.py`)

- Audits up to 200 historical turns from `agent_habit_log`.
- Identifies the agent's weakest dimension (Tone, Empathy, or Clarity) and clusters recurring coaching keywords.
- Generates a targeted **Micro-Habit Practice Card** with a template, turn goal, and measurable success criterion.

3.5 FAISS Vector Knowledge Base (`server/knowledge_base.py`)

- Vectorizes enterprise policies and FAQs using `sentence-transformers/all-MiniLM-L6-v2` into dense 384-dimensional vectors.
- Queries via `faiss.IndexFlatL2` in **<10ms** for 1-click policy injection into agent draft responses.

4. Backend REST API Specification (Port 5000)

Method	Route	Description & Key Parameters
`GET`	`/api/status`	Returns engine health, active LLM provider (`groq` / `claude`), and knowledge base status.
`GET`	`/api/sessions`	Returns active sessions with customer name, plan tier, turn counts, and latest sentiment.
`POST`	`/api/session/new`	Thread-safe creation of new tickets (`TK-<id>`). Accepts `name`, `email`, `plan`, `initial_message`.
`GET`	`/api/session/<id>`	Returns full ticket details, customer metadata, and turn history.
`DELETE`	`/api/session/<id>`	Deletes session and cascades deletion of turns from SQLite.
`POST`	`/api/session/reset`	Resets turns for session. Returns `400` if missing `session_id`, `404` if not found.
`POST`	`/api/coach`	Unified coaching turn processing. Evaluates customer message, agent draft, compliance, burnout, and momentum in one shot.
`GET`	`/api/session/<id>/burnout`	Returns current burnout index (0–100), risk tier, and recommended supervisor action.
`GET`	`/api/session/<id>/momentum`	Returns outcome prediction, confidence percentage, and estimated turns until resolution.
`GET`	`/api/agent/habits`	Analyzes agent history and generates personalized micro-habit coaching card.
`POST`	`/api/knowledge/search`	Performs FAISS dense vector search over FAQs and policy documents.
`GET`	`/api/history`	Returns complete session and turn log ordered chronologically newest-first.

Sample Payload: POST /api/coach

```
// Request
{
  "session_id": "TK-1",
  "customer_message": "I was charged twice for invoice #104.",
  "agent_message": "I apologize for the error. Let me inspect your billing profile and process the refund.",
  "agent_id": "agent_anshu"
}

// Response (Status: 200 OK, Latency: 0.38s)
{
  "analysis": {
    "sentiment": "negative",
    "urgency": "medium",
    "intent": "billing_dispute",
    "churn_risk": "low",
    "escalate": false,
    "key_issues": ["charged twice", "invoice #104"]
  },
  "feedback": {
    "tone_score": 9,
    "empathy_score": 9,
    "clarity_score": 9,
    "coaching_tip": "Good acknowledgment. Specify refund timeline (3-5 business days).",
    "knowledge_suggestion": "Duplicate charges policy clause found."
  },
  "compliance": { "violation": false, "issue": "", "suggestion": "" },
  "burnout": { "burnout_index": 18.2, "burnout_risk": "low" },
  "momentum": { "outcome_prediction": "resolution", "confidence": 92, "turns_until_outcome": 1 }
}
```

5. Persistence Layer & Database Schemas

The system utilizes SQLite3 (`sessions.db`) managed via `server/database.py` :

```
-- Sessions Table
CREATE TABLE IF NOT EXISTS sessions (
    id             TEXT PRIMARY KEY,
    title          TEXT,
    customer_json  TEXT,
    created_at     TEXT,
    updated_at     TEXT,
    last_sentiment TEXT DEFAULT 'neutral',
    last_urgency   TEXT DEFAULT 'low'
);

-- Turns Table (Cascading Deletion)
CREATE TABLE IF NOT EXISTS turns (
    id               INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id       TEXT NOT NULL,
    customer_message TEXT,
    agent_message    TEXT,
    result_json      TEXT,
    timestamp        TEXT,
    FOREIGN KEY (session_id) REFERENCES sessions(id) ON DELETE CASCADE
);

-- Agent Habit Log Table (With Foreign Key Constraint & Safe Migration)
CREATE TABLE IF NOT EXISTS agent_habit_log (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    agent_id      TEXT NOT NULL DEFAULT 'default_agent',
    session_id    TEXT NOT NULL,
    tone_score    REAL,
    empathy_score REAL,
    clarity_score REAL,
    coaching_tip  TEXT,
    timestamp     TEXT,
    FOREIGN KEY (session_id) REFERENCES sessions(id) ON DELETE CASCADE
);
```

> **Automatic Zero-Downtime Migration Routine**
>
> In `server/database.py` , `init_db()` executes `PRAGMA foreign_key_list(agent_habit_log)` . If the FK constraint is missing from earlier installs, it creates `_agent_habit_log_new` with the foreign key, copies all existing data, and renames the table cleanly without dropping historical agent turns.

6. Frontend Architecture & Full Module Tour

Page / Component	Route	Core Capabilities
`LandingPage.jsx`	`/`	Product showcase, interactive scenario simulators (Double Charge, Delivery Tracking, Hindi Regional Query, Resolution), ROI calculator.
`ConversationCanvas.jsx`	`/workspace`	Live chat timeline with agent avatars, message composer, suggested reply injection buttons, and multi-turn alignment.
`CopilotSidebar.jsx`	`/workspace`	3-column assistant panel: Tone/Empathy/Clarity radial score rings, live coaching tips, compliance alerts, and 1-click vector KB cards.
`Tickets.jsx`	`/tickets`	Full lifecycle management supporting all 4 statuses (`Open` , `Pending` , `Resolved` , `Closed`), admin cross-account filtering, and Firestore sync.
`LiveQueue.jsx`	`/queue`	Live inbound queue with real-time SLA countdown timers, urgency badges, and 1-click ticket claiming.
`Analytics.jsx`	`/analytics`	Interactive analytics with 7d vs 30d toggling, hourly volume heatmaps, and CSAT / resolution trends.
`Reports.jsx`	`/reports`	Role-governed audit reports (Admins see global company data; Agents see individual statistics) and CSV export.
`AgentPerformance.jsx`	`/performance`	Dynamic leaderboard featuring an adaptive 3-tier Olympic podium, burnout risk chips, and AI micro-habit practice cards.
`TeamManagement.jsx`	`/team`	Roster of team members with role assignments (`Admin` , `Supervisor` , `Agent`), department filtering, and online status toggles.
`Customers.jsx`	`/customers`	Customer directory CRM with plan tiers, MRR/ARR values, lifetime value, and historical ticket logs.
`Settings.jsx`	`/settings`	Profile updating (`displayName`), in-profile password updates, password reset link dispatch, AI inference configuration.
`AuthPage.jsx`	`/auth`	Secure Firebase Authentication supporting sign-up, sign-in, persistent sessions, role management, and password reset email dispatch.

7. Verification, Testing & Bug Fix Audit (v2.1.0)

OmniDesk Copilot includes an automated smoke test suite verifying all coaching intelligence engines:

```
python test_novel_features.py
============================================================
  omniDesk-copilot — Coaching Intelligence Tests
============================================================
[1] Agent Burnout Detector ......... PASS (burnout_index: 69.7)
[2] Conversation Momentum Forecaster PASS (confidence: 95%, turns: 1)
[3] Micro-Habit Coach .............. PASS (weakest: empathy)
============================================================
  ALL 3 ACTIVE FEATURE SMOKE TESTS PASSED
============================================================
```

Summary of 6 Resolved Bugs

#	Severity	File & Function	Issue & Resolution
1	High	`coach.py` (`_call_llm`)	Function fell through if provider was neither 'groq' nor 'claude', returning `None` and corrupting JSON scores. Added explicit `else: raise ValueError(...)`.
2	High	`database.py` (`agent_habit_log`)	Lacked `FOREIGN KEY (session_id) REFERENCES sessions(id) ON DELETE CASCADE`. Added FK constraint and safe migration routine in `init_db()`.
3	Medium	`app.py` (`session_counter`)	Increment was unprotected, risking ID collision under concurrent requests. Wrapped in `threading.Lock()`.
4	Medium	`app.py` (`reset_session`)	Returned misleading success with null ID when session_id was missing. Added guards returning HTTP `400` and `404`.
5	Low	`models.py` (`CoachingFeedback`)	Lacked dictionary access method, risking `AttributeError`. Added `.get(key, default)` method for full dict compatibility.
6	Low	`app.py` (`_bootstrap_from_db`)	Burnout baselines and momentum signals were lost on server reboot. Added replay of stored turns through burnout and momentum engines on startup.